Research Report: Toy Blockchain and Ledger Simulator

Golang Developer Assessment – Backend Engineering Internship
July 2026

Introduction

This report documents experiments run against my own implementation (`toyblockchain`), a single-process, proof-of-work blockchain and ledger written in Go, plus a short design write-up and a discussion of how this toy relates to production blockchains. Every number and every terminal transcript in this report was produced by actually running the code in this repository on my own machine, not estimated or reconstructed afterwards.

The implementation covers all required functional requirements (block structure, genesis block, deterministic hashing, transactions and a ledger, bounded proof-of-work mining, full-chain validation with tamper detection, a command-line interface, and JSON persistence), and also implements all five optional stretch goals: digital signatures, a Merkle root, concurrent mining, automatic difficulty retargeting, and fork resolution. Since the first draft of this report, two further pieces have been added and are documented in Section 6: **Merkle inclusion proofs** (light-client-style verification that one transaction belongs to a block, without the rest of the block) and a **test suite for the `cli` package**, so every package in the repository now has direct test coverage — 66 tests in total. The implementation also adds two pieces of robustness that were not explicitly required but that a CLI tool needs to survive being interrupted mid-mine or run twice against the same file: atomic saves and a process-level file lock.

1. Tamper-evidence experiment

Setup. I built a small honest chain using the CLI (wallet creation, a faucet grant, a signed transfer, and mined blocks), confirmed it validates, then edited the persisted chain file directly on disk with a text editor, simulating an attacker with raw file access.

Before tampering, the chain loads and validates cleanly:

```
PS D:\toyblockchain> go run . -data "data/chain.json" -wallets "data/wallets.json" validate
Loaded chain from data/chain.json (4 blocks).
Chain is VALID (4 blocks)
```

Tamper. I made a backup copy of the valid file, then opened `data/chain.json` in Notepad and hand-edited one transaction field inside an already-mined block, without touching that block's stored hash field and without re-mining anything.

After tampering, simply starting the program again fails immediately:

```
PS D:\toyblockchain> go run . -data "data/chain.json" -wallets "data/wallets.json"
Error: loaded chain is invalid: block 2 invalid: stored hash does not match recomputed hash
exit status 1
```

Why this happens, precisely. `chain.Load` calls `Chain.Validate` before ever handing the loaded chain back to the caller, so a tampered file is rejected at load time. `Chain.Validate` recomputes each block's hash from its current on-disk fields and compares that recomputed hash against the hash field stored in the same block. Editing a transaction amount changes the JSON payload that ultimately feeds SHA-256, and by the avalanche property of a cryptographic hash function this produces a completely different digest, caught at exactly the block I altered.

Now that transactions are summarised by a Merkle root, there are **two independent checks, not one**. `Chain.Validate` recomputes a block's Merkle root from its stored transactions before it even looks at the block hash, and rejects the block if the recorded root doesn't match:

```
testChain.Blocks[2].Transactions[0].Amount = 999999
err := testChain.Validate()
// -> &ValidationError{BlockHeight: 2, Reason: "stored Merkle root does not match transactions"}
```

This is `TestTransactionTamperingBreaksMerkleRoot`. A more careful attacker who also patches the block's recorded Merkle root to match the edited transaction (but, again, without re-mining) is caught one check later, by the ordinary hash-recomputation check, since the block hash is computed over the Merkle root and a changed root means a changed hash:

```
testChain.Blocks[2].MerkleRoot = block.CalculateMerkleRoot(testChain.Blocks[2].Transactions)
err := testChain.Validate()
// -> &ValidationError{BlockHeight: 2, Reason: "stored hash does not match recomputed hash"}
```

This is `TestTamperedMerkleRootRejected`. And, as before, an attacker patient enough to recompute both the Merkle root and the block hash is still caught by the difficulty check, because the recomputed hash was not actually mined and essentially never satisfies the required number of leading zero hex digits by chance (`TestTamperedDifficultyRejected` demonstrates the equivalent scenario for a tampered difficulty value). Whichever layer catches it, the conclusion is the same: the only way to make a tampered block internally consistent is to redo the proof-of-work for it, which then breaks the `prev_hash` link to the next block, forcing every later block to be re-mined too — the same cost that makes real blockchains tamper-resistant (see the discussion questions below).

Both scenarios are exercised as automated regression tests, so this is checked on every run, not only when I remember to try it by hand.

2. Difficulty versus effort experiment

Setup. I wrote a small standalone driver (kept outside the shipped CLI, since it is not part of the deliverable) that mines fresh blocks directly through the `block` package's `Mine` method at difficulties 1 through 6, five trials per difficulty level, with a 30-second timeout per trial, using a single mining worker so the numbers are comparable to a simple sequential search.

Results, averaged over the trials that completed within the timeout:

Difficulty (leading hex zeros)	Avg. attempts	Avg. time	Trials completed
1	20	0.22 ms	5 / 5
2	42	~0 ms	5 / 5
3	1,635	2.17 ms	5 / 5
4	134,910	154.69 ms	5 / 5
5	63,976	73.43 ms	5 / 5
6	14,071,708	15.90 s	3 / 5

An honest note on difficulty 5. Read literally, the average for difficulty 5 is lower than the average for difficulty 4, which looks like it contradicts the trend. It doesn't: the number of attempts needed to find a hash with a given number of leading zero hex digits follows a geometric distribution, which has high variance, and five trials is a small sample. Difficulty 4's five trials happened to include one unusually expensive run; difficulty 5's five trials happened to land close together on the low side. The difficulty 6 row makes the same point from the other direction: two of the five trials hit the 30-second timeout before finding a nonce at all, which is itself strong evidence of how much the tail of the distribution grows, even though the three that did complete average out to a number consistent with the overall trend.

Trend. Despite the sampling noise, the shape is unmistakable: this is not linear, it grows roughly exponentially. Going from difficulty 1 to difficulty 6 multiplies the expected work by roughly 700,000x, not 6x. Each hex character of a SHA-256 digest is effectively uniformly random over 16 values, so requiring one more leading zero hex digit divides the probability of any single attempt succeeding by 16 and multiplies the expected attempts needed by roughly 16 — an exponential function of difficulty (on the order of 16^N), not a polynomial or linear one. This is the whole point of proof-of-work as a rate-limiting mechanism: a small, linear increase in the agreed difficulty translates into an exponential increase in the cost of finding a valid block.

3. Concurrent mining

Mining now searches the nonce space across multiple goroutines rather than a single sequential loop, using `-mining-workers` (default: `runtime.NumCPU()`). A real session on a 4-core machine, immediately after creating a fresh chain:

```
> faucet 458a00afa186b42e2626f28c4a0866733469a9d3 500
Queued faucet grant: 458a00afa186b42e2626f28c4a0866733469a9d3 receives 500
> mine
Mined block #1
  Hash:       000245e416120786046ac95271933405e6bf2aaea18d91ddbdc1620cad5bea3b
  Nonce:      13834
  Attempts:   13304
  Difficulty: 3
  Workers:    4
  Time:       7.7072ms
```

Two things stand out here and are worth being honest about rather than glossing over. First, `Attempts` (13,304) is lower than `Nonce` (13,834), which looks odd at first — with a single worker the two are always equal, since attempt n always tries nonce n-1. With four workers racing, the winning nonce can be higher than the attempt count because the four workers are exploring disjoint, interleaved nonce ranges concurrently (worker 0 tries 0, 4, 8, …; worker 1 tries 1, 5, 9, …; and so on), and the winning worker doesn't have to be the one that made the most attempts — a later-numbered nonce found early in one worker's private sequence can still win the race against an earlier-numbered nonce that a different worker hasn't reached yet. `Attempts` is an exact count (via `atomic.Uint64`, incremented by every worker) of total hash computations actually performed across all workers combined, which is the number that matters for measuring real work done; `Nonce` is just the specific value that happened to satisfy the target. Second, this particular block (13,304 attempts at difficulty 3) took noticeably more attempts than the single-worker table above suggests is typical at difficulty 3 (around 1,635) — again, this is exactly the same high-variance geometric distribution from Section 2, not a bug; some individual mining runs are simply unlucky.

Why this is safe. Each worker computes a candidate hash against its own local copy of the block (`candidate := *b`) rather than mutating the shared block directly, and only the single winning result is ever written back to the real block, after all workers have stopped. I confirmed there is no data race with `go test ./... -race`, which passed cleanly across all 39 tests that existed when concurrent mining was added. One honest caveat: later in development the MinGW gcc on my Windows machine turned out to be 32-bit-only and can no longer *build* the race-instrumented runtime at all (`cc1.exe: sorry, unimplemented: 64-bit mode not compiled in` — a toolchain limitation, not a detected race), so the newest tests added since then have been run without `-race` on this machine. The concurrent mining code itself has not changed since it was race-verified, and none of the code added afterwards (Merkle proofs, retargeting, fork resolution, CLI tests) spawns goroutines, but I am flagging the gap rather than silently letting the earlier claim stand for code it never covered.

Why genesis is the one exception. If genesis were mined with multiple workers, whichever worker happened to win the race would be essentially random, so the genesis nonce (and therefore the genesis hash) would not be reproducible across different machines or core counts from the difficulty alone. `NewGenesisBlock` always pins genesis mining to exactly one worker to keep it fully deterministic, which is exercised directly by `TestGenesisDeterministicAcrossWorkerCounts`.

4. Automatic difficulty retargeting

Beyond the manual `setdifficulty` schedule, a chain can now be told to retarget its own difficulty automatically toward a target block time. I created a chain with `-retarget=true -retarget-interval=5 -target-block-time=5s -min-difficulty=1 -max-difficulty=8` and mined six blocks in quick succession (well under the 5-second-per-block target, since difficulty 3 mines in milliseconds on this machine):

```
> retarget
Automatic difficulty retargeting:
```

```
  Enabled:              true
  Interval:             5 blocks
  Target block time:    5 seconds
  Minimum difficulty:   1
  Maximum difficulty:   8
  Next retarget height: 6
  Next block difficulty: 3
...
(five blocks mined, all difficulty 3, all in a few milliseconds each)
...
> retarget
Automatic difficulty retargeting:
  ...
  Next retarget height: 6
  Next block difficulty: 4
> mine
Mined block #6
  Hash:       0000be9f412af977a774e46202b28405bb39fad622909cfa2a18ed616672b654
  Nonce:      4103
  Attempts:   4338
  Difficulty: 4
  Workers:    4
  Time:       2.1946ms
> validate
Chain is VALID (7 blocks)
```

This is exactly the intended behaviour: the first five real blocks (heights 1-5) were mined far faster than the 5-second target, so once the retarget window closed at height 6 the policy raised the difficulty from 3 to 4 for that block onward, and `retarget` correctly previewed "Next block difficulty: 4" before block 6 was actually mined. `policy` (the manual schedule) still correctly showed only the original `from height 0: difficulty 3` rule, since the difficulty-4 change here came entirely from the automatic policy, not a manual `setdifficulty` call — the two mechanisms are deliberately kept visible as separate pieces of state, and `Chain.Validate` checks the combined result of both against every block's recorded difficulty.

Along the way I also hit, and want to record honestly, an ordinary concurrency mistake of my own rather than a bug in the tool: I started a second `go run .` against the same `data/chain.json` while the first interactive session was still open, and got exactly the "chain file is locked by another process" error the file lock is supposed to produce. That was working as intended; the fix was simply to close the first session (or remove the stale `.lock` file if a session had genuinely crashed) rather than a code change.

5. Fork resolution

To exercise fork resolution I built two independent, unrelated-looking but same-policy chains from scratch and then reconciled them.

Chain A (`current-fork.json`, difficulty 1, 2 workers): one faucet grant to `alice`, one block mined, reported accumulated work 32 (`16^1 + 16^1`, one unit for genesis and one for the mined block, both at difficulty 1).

Chain B (`candidate-fork.json`, difficulty 1, 4 workers, otherwise the same policy): one faucet grant to `bob`, one block, then a second faucet grant to `carol`, a second block — reported accumulated work 48 (`16^1 × 3`, three blocks total including genesis).

Reconciling them from Chain A's side:

```
> work
Accumulated work: 32
> resolvefork data/candidate-fork.json
Fork resolved. Stronger chain adopted.
  Common ancestor: block 0
  Old height:      1
  New height:      2
  Old work:        32
  New work:        48
```

```
  Pending kept:    0
  Pending dropped: 0
> work
Accumulated work: 48
> validate
Chain is VALID (3 blocks)
```

Chain A correctly identified that Chain B had strictly more accumulated work (48 > 32), confirmed both chains shared the same genesis block hash (`Common ancestor: block 0`), and replaced its own single mined block with Chain B's two mined blocks. `print` afterward showed Chain A now contains `bob`'s and `carol`'s transactions from Chain B, not `alice`'s original transaction from Chain A's own losing block — that transaction is gone from the adopted chain (it was never included in Chain B), which is the expected, if slightly unforgiving, behaviour of "the stronger chain wins outright" rather than a merge of both histories. No pending transactions were involved in this run, but `TestResolveForkRevalidatesPendingTransactions` covers the case where losing-chain and winning-chain pending pools both have queued transactions: they are combined, deduplicated, and each individually re-validated against the new chain, so a transaction that's still valid against the winning history survives and one that no longer is (for example because its nonce or balance no longer lines up) is dropped rather than silently corrupting the pending pool.

Why work, not just height. `TotalWork` sums `16^difficulty` per block rather than counting blocks, because a chain with more blocks at a lower difficulty can represent less actual computation than a shorter chain at higher difficulty. `TestResolveForkRejectsEqualOrWeakerCandidate` checks that a same-or-lower-work candidate is rejected even if it happens to be taller. `TestResolveForkRejectsDifferentGenesis` and `TestResolveForkRejectsInvalidCandidate` check the two "safety" gates before work is even compared: the candidate must descend from the same genesis block and must itself pass full validation, so `resolvefork` can't be used to adopt an internally-tampered chain just because it claims a large work total.

6. Merkle inclusion proofs (new since the first draft)

The first draft of this report listed "no Merkle proof / light-client verification" as an honest remaining gap and sketched how it would be added. That gap has now been closed, largely along the lines of the original sketch, in `block/merkle_proof.go`.

Generation. `GenerateMerkleProof(transactions, index)` walks exactly the same pairing structure as `CalculateMerkleRoot` — including duplicating the last hash whenever a level has an odd count, which must match bit-for-bit or proofs built here would never verify against roots computed there — but instead of discarding intermediate hashes, it records at each level the sibling hash of the node on the path from the target transaction up to the root, plus whether that sibling sits on the left or the right (a `ProofStep{Hash, Left}`), which is needed to hash each pair back together in the correct order during verification. Out-of-range indexes are rejected with a clear error rather than a panic.

Verification. `VerifyMerkleProof(txHash, proof, root)` starts from a single transaction's own hash and folds in each proof step's sibling — left or right as recorded — recomputing one SHA-256 per level, then compares the result to the known root. The whole check touches `log2(n)` hashes instead of all n transactions, which is precisely what lets a real chain's light clients verify a payment without downloading full blocks.

Tests. Five dedicated tests cover the property that matters and the ways it should fail: a valid proof verifies (`TestMerkleProofVerifiesInclusion`); a proof for one transaction does not verify a different one (`TestMerkleProofFailsForWrongLeaf`); a valid proof fails against the wrong root (`TestMerkleProofFailsAgainstWrongRoot`); out-of-range indexes error cleanly (`TestMerkleProofIndexOutOfRange`); and the single-transaction edge case, where the proof is empty and the root is the transaction's own hash, works (`TestMerkleProofSingleTransaction`).

A deliberate scope decision. This is a library-level API in the `block` package with no CLI command in front of it, because this toy has no light client that would consume a proof — the CLI always holds the full chain locally, so a `prove <tx>` command would demonstrate the machinery without anyone needing its output.

The API is where the real substance is, and it is fully tested.

7. Test coverage of the cli package (new since the first draft)

The `cli` package was originally the one package without its own tests — command dispatch was only exercised manually. `cli/cli_test.go` now runs 11 tests through the same dispatch path the interactive REPL uses, against temporary chain and wallet files, covering: the help listing (`TestHelpPrintsCommandList`), unknown-command rejection (`TestUnknownCommandRejected`), wallet creation and listing (`TestWalletCreateAndList`), the full faucet → mine → balances happy path (`TestFaucetMineAndBalances`), signed sends being queued (`TestSendSignsAndQueuesTransaction`) and unknown-wallet sends being refused (`TestSendUnknownWalletRejected`), the manual difficulty schedule (`TestSetDifficultyAndPolicy`), the rule that retarget configuration is only changeable before mining starts (`TestRetargetConfigOnlyBeforeMining`), tamper reporting surfacing through `validate` (`TestValidateReportsTamperedChain`), the pending pool and explicit save (`TestPendingAndSave`), and work reporting plus fork resolution end-to-end (`TestWorkAndResolveFork`). With these, the full suite stands at 66 tests across all four testable packages (`block`, `chain`, `ledger`, `cli`), and `go test ./...`, `go vet ./...`, and `gofmt` are all clean.

8. Design write-up

Hashing scheme. Each block's hash is `SHA-256(JSON(payload))`, where payload is an explicitly-ordered struct with these fields, in this exact order: `height`, `timestamp`, `merkle_root`, `prev_hash`, `nonce`, `difficulty`. The block's own hash field is excluded, since it is the output of the computation. `encoding/json` serialises struct fields in declaration order, never map order, which makes the output deterministic without any extra bookkeeping.

Merkle root instead of a raw transaction list. `CalculateMerkleRoot` hashes each transaction individually with SHA-256, then repeatedly hashes adjacent pairs together (duplicating the last hash when a level has an odd count) until one root remains. I chose to fold this into the block hash payload as a single `merkle_root` string rather than hashing the full transaction list directly, both because it's what real chains do and because it means a block's hash no longer grows more expensive to compute as the transaction count grows (each block hash computation is a constant number of SHA-256 calls over the root and a handful of scalar fields, regardless of how many transactions built that root) — the transaction-dependent work is isolated to `CalculateMerkleRoot` itself, which only needs to be recomputed when transactions actually change. Section 6 describes how the same tree structure now also yields per-transaction inclusion proofs.

Signature scheme. Signatures are computed over a narrower `transactionPayload` struct (`sender`, `recipient`, `amount`, `nonce`) rather than the whole `Transaction`, so the public key and signature fields are never inputs to what the signature protects. Ed25519 was chosen over RSA/ECDSA for its fixed small key/signature sizes and lack of parameter choices to get wrong.

How validation guarantees integrity across the whole chain. `Chain.Validate` walks blocks from genesis to tip and, for every block, checks: (1) the block's recorded difficulty matches what the combined manual-schedule-plus-retarget policy says that height should require, (2) the block's recorded Merkle root matches a fresh recomputation from its stored transactions, (3) recomputing the hash matches the stored hash, (4) the stored hash actually satisfies the required difficulty target, (5) structural checks (genesis fields, or height/timestamp/prev-hash linkage for later blocks), and (6) every transaction replays cleanly through a running ledger (signatures, nonces, balances). Any failure returns immediately with the offending block's height and a human-readable reason. Because each block's hash is computed over its own `prev_hash`, and the next block stores that hash as its own `prev_hash`, every block is cryptographically bound to the one before it — tampering with block k is only "free" if you're willing to redo the proof-of-work for block k and every block after it, which Section 2's timings show grows exponentially expensive with difficulty.

9. Discussion questions

How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy? Locally, "trivial" only in the sense that nothing stops me from editing the JSON file directly, as Section 1 demonstrates — but even locally, a tamper that survives the Merkle-root and hash-recomputation checks still gets caught by the difficulty check, because faking a valid hash for new content requires re-mining. The real difference in a production chain isn't that re-mining one block is somehow computationally harder there than here — it's the same computation regardless of who runs it. The difference is that re-mining one block isn't enough in a live network: every peer independently holds a copy of the chain and only accepts the chain with the most accumulated work it sees (the same `TotalWork` comparison Section 5's fork resolution implements locally). To rewrite an old block, an attacker has to re-mine that block and every block after it fast enough to produce an alternative chain that overtakes the one the rest of the honest network is still extending in real time — a race against the combined hashing power of every other honest participant, not a one-off computation on a single laptop with no deadline. My toy's `resolvefork` command implements the comparison rule (more work wins) but there is still no network of peers automatically running that comparison against each other in real time; a real network's tamper-resistance comes from that live race, not merely from the work-comparison rule existing.

Proof-of-work is one way to decide who may add the next block. Name at least one alternative and give one advantage and one drawback versus proof-of-work. Proof-of-stake (used by, for example, Ethereum since its 2022 "Merge") selects the next block proposer based on how much cryptocurrency they have locked up as stake, rather than how much computation they have burned searching for a nonce. Advantage: it does not require racing large amounts of real-world energy, so it is far cheaper to run and doesn't scale its environmental footprint with network security the way proof-of-work does. Drawback: the cost of attacking the network is now denominated in the network's own token rather than an external physical resource, which raises subtler questions such as "nothing at stake" style attacks and wealth concentration circularly affecting block-producing power.

List three concrete ways your toy differs from a production blockchain. Pick one and sketch how you would add it. With signatures, a Merkle root (now including per-transaction inclusion proofs), concurrent mining, retargeting, and manual fork resolution all implemented, the honest remaining gaps are:

1. **No peer-to-peer network.** `resolvefork` implements the "more work wins" comparison rule, but a human still has to manually hand one chain file to another process — there is no automatic peer discovery, block gossip, or continuous chain-selection happening between running processes.

2. **No encrypted wallet storage.** Private keys sit in plaintext JSON, fine for a local CLI demo but not something to build a real wallet on.

3. **No real finality or confirmation depth.** A block is "final" here the instant it is mined; there is no notion of waiting k confirmations before treating a payment as settled, which real chains need precisely because a stronger competing fork (Section 5) can still arrive and orphan recent blocks.

Sketch: adding a minimal peer-to-peer layer. (The first draft of this report sketched Merkle proofs here; since those are now implemented — Section 6 — the sketch below covers the largest remaining gap instead.) I would keep the consensus rule exactly as it is — `ResolveFork`'s "valid, same genesis, strictly more work" check is already the chain-selection rule a node needs — and add a thin networking layer around it: each node runs a small HTTP or TCP listener (Go's `net/http` is enough) exposing three endpoints: announce a newly mined block, serve the full chain on request, and exchange known-peer lists for rudimentary discovery. On receiving an announced block, a node tries to append it; if the block doesn't extend its tip (heights or hashes don't line up), it fetches the announcer's full chain and feeds it through the existing `ResolveFork` path, so a stronger chain is adopted and a weaker one rejected using code that is already written and tested. Pending transactions would gossip the same way, deduplicated by the signature-plus-nonce identity the pending pool already enforces. The genuinely new work is not consensus but plumbing: peer lifecycle, retry/backoff, and not trusting anything a peer sends until it has passed the same full validation local files already go through — which is why I judged it the right thing to sketch rather than rush, having prioritised making the local consensus rules correct and tested first.

10. Other behaviour worth recording

Mining failure keeps pending transactions. Running with a deliberately tiny attempt budget after raising the difficulty makes mining fail as expected, and the queued transaction is not silently lost:

```
> mine
Error: maximum mining attempts reached
> pending
  FAUCET -> 3a47f5505930e7b01d869bb5664ad38adbe91983 : 1 (nonce 0)
```

This matches `TestMiningFailureKeepsPendingTransactions`: a failed `MineBlock` call must not remove anything from the pending pool and must not append a half-mined block.

The file lock does what it says. Starting a second CLI instance against a data file already locked by a running process fails immediately with a clear error rather than silently loading stale state:

```
Error: chain file is locked by another process
exit status 1
```

Covered by `TestFileLockRejectsSecondProcess`, and (as noted in Section 4) reproduced by accident during the retargeting session when I genuinely forgot a previous session was still open.

Sources

General blockchain concepts referenced above (proof-of-work as a difficulty-scaled search problem, the longest/heaviest-valid-chain rule, and Merkle trees for compact transaction summaries and inclusion proofs) follow the design first described in Satoshi Nakamoto's Bitcoin whitepaper, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008). The description of Ethereum's move to proof-of-stake follows publicly documented information about "The Merge" from the Ethereum Foundation's own documentation and blog posts. No text from either source is reproduced here; all explanations above are written in my own words based on general, publicly available knowledge of how these systems work.